Checkers

SyoSil, 2026

SyoSil ApS © 2026

Contents

- Setting the scene
- Types of checkers
- What can affect a checker
- Reference model
- Score board
- Transaction Checkers
- Timing checkers
- Where to apply checkers

SyoSil ApS © 2026

Setting the Scene

SyoSil ApS © 2026

Setting the Scene

- As discussed earlier then:
  - Stimuli
  - Coverage
  - Checkers

  Must be perpendicular to each other for the computer to write the tests

- Thus, checkers are forming a very important part of the functional verification as they **ARE** the verification.
  - Stimuli is just the input
  - Coverage is just there to observe that we have visited interesting parts of the state space
  - Checkers verify the intended functionality

SyoSil ApS © 2026

Types of Checkers

SyoSil ApS © 2026

Types of Checkers

- Reference model
  - A reference model models the functionality of the design and provides model checks which is used to check the behavior of the RTL design
- Score board
  - Typically compares expected transactions with actual transactions
- Transaction Checkers
  - Used e.g. check the integral parts of a transaction or stream of transactions
- Timing checkers
  - Checks the temporal behavior of events

SyoSil ApS © 2026

What can affect a Checker?

SyoSil ApS © 2026

What can Affect a Checker

- **Static** information for a given simulation
  - Does not change during the simulation of a test
  - Parameters
  - Control knobs
- **Dynamic** information within a single simulation
  - Does change during the simulation of a test
  - Configuration settings
    - ENV config
    - Registers
  - Clock frequency
  - Reset
  - Stimuli
  - Events

Thus, the checkers will need this information as they need to adapt accordingly!

SyoSil ApS © 2026

Reference Model

SyoSil ApS © 2026

Reference Model

- Get all relevant inputs and computes the output
  - Functionality only, no timing
  - If timing, then the model is typically CA (Cycle Accurate)
- Typically operates on transactions
  - Pixels
  - Frames
  - Buffers
- For PyUVM TBs then
  - Typically written in Python, C or Verilog
- For SystemVerilog UVM
  - Also, in SystemC TLM, Matlab




```
class adder_reference_model:
    __init__(..):

    def compute_output(self, a, b):
        return a+b
```

Reference Model Example

- Implements reference for addition of to numbers

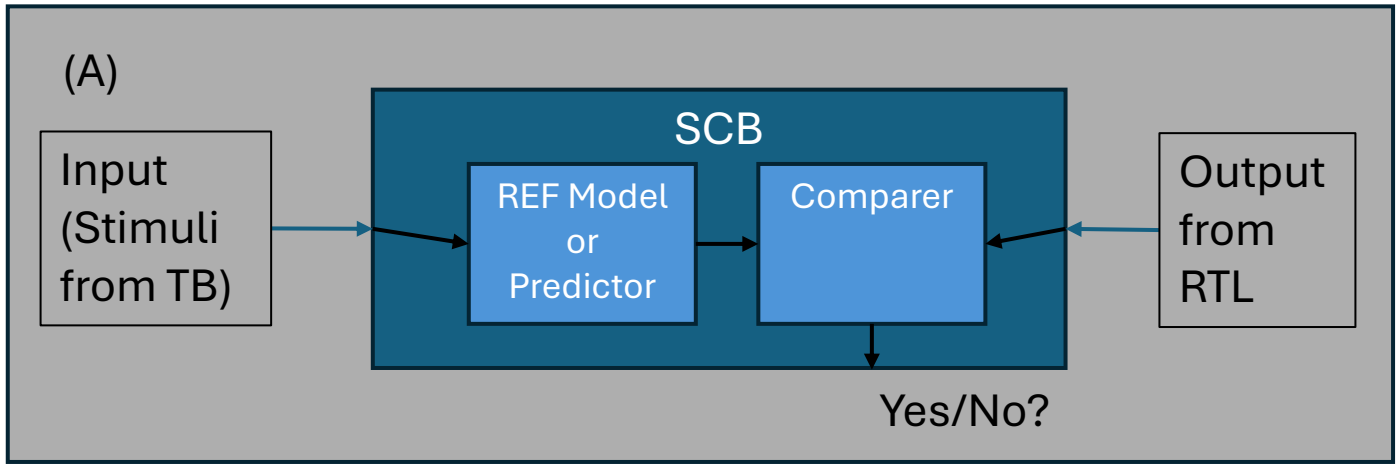
SyoSil ApS © 2026

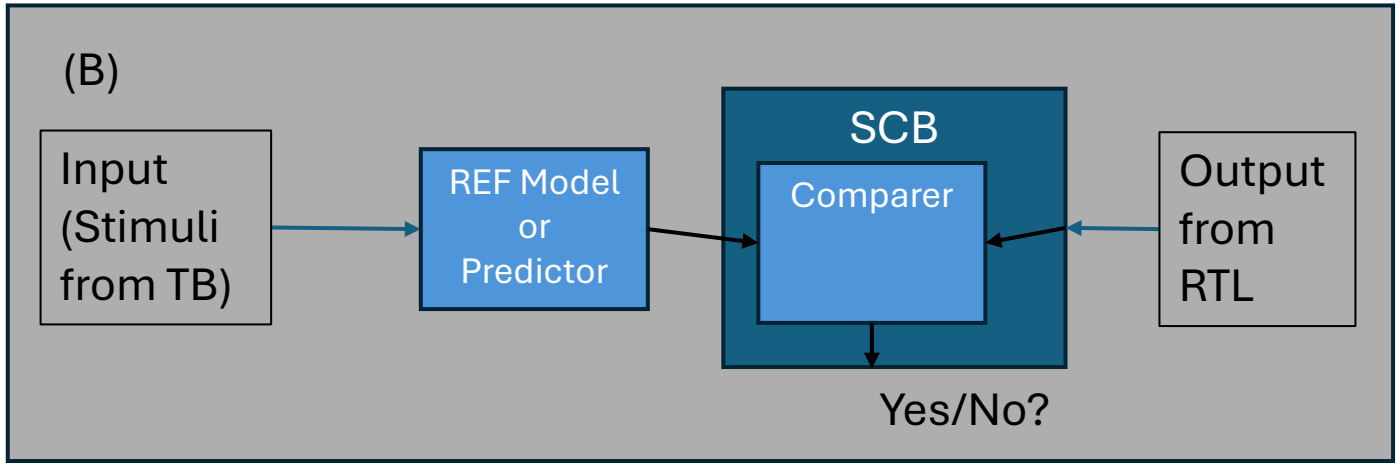
Score board

SyoSil ApS © 2026

Score board

- Typically compare transactions
- Two flavors
  - (A) Score board consists of
    - Reference Model or Predictor
    - Comparer
  - (B) Or
    - Comparer (Architectural separation)
    - Why is this a good idea:
      - https://www.syosil.com/images/resources/dvcon15_us_andersen_jensen_steffensen.pdf
      - Easy maintenance
      - Supports multiple models in parallel
      - Around 15% of the TBs source code can be reused





SyoSil ApS © 2026

Score board

- Can be instantiated multiple times
- In a PyUVM TB then typically the uvm_sequence_items are compared
- A part of the verification methodology is to decide WHAT should go to a score board
- Examples
  - Register reads
  - Output transactions

```python
from pyuvm import uvm_component, uvm_analysis_export, uvm_error

class Scoreboard(uvm_component):
    def build_phase(self):
        super().build_phase()
        self.ap = uvm_analysis_export("ap", self)
        self.expected_data = []

    def write(self, item):
        if self.expected_data:
            expected_item = self.expected_data.pop(0)
            if item != expected_item:
                self.uvm_error("Scoreboard", f"Mismatch: Got {item},
                                                expected {expected_item}")
            else:
                self.uvm_info("Scoreboard", f"Matched: {item}", UVM_LOW)
        else:
            self.uvm_error("Scoreboard", f"Unexpected item received: {item}")

    def add_expected_data(self, data):
        self.expected_data.append(data)
```

SCB Example

- With architectural separation
- REF model MUST compute expected results and call `add_expected_data` to add expected data

SyoSil ApS © 2026

Transaction Checkers

SyoSil ApS © 2026

Transaction Checker

- A score board is a transaction checker
  - Sometimes this is not enough!

```
class DataChecker(uvm_subscriber):
    def build_phase(self):
        super().build_phase()
        self.data_under_10 = True

    def write(self, seq_item):
        if seq_item.data >= 10:
            self.data_under_10 = False
            self.uvm_error("DataChecker",
                           f"Data value {seq_item.data} is not less than 10")
        else:
            self.uvm_info("DataChecker",
                          f"Data value {seq_item.data} is less than 10", UVM_LOW)
```

Transaction Checker Example

- Checks that data is less than 10 and flags an error if it is not

- Maybe need for checking
  - Content of transaction
    - E.g. Inside the monitor of a UVC
  - Stream of transactions
    - E.g. Inside the monitor of a UVC
    - Or another checker implemented as a UVM component
      - Get stream of transactions
      - Perform checks

SyoSil ApS © 2026

Timing Checkers

SyoSil ApS © 2026

Timing Checkers AKA Assertions

- Checks the timing of events
- Events can be
  - Signal behavior over time
  - Specific high-level events over time
- Examples
  - When signal A is asserted then after 5 clks signal B should be asserted for 1 clk.
  - An IRQ event should occur when the DUT finishes processing the input

SyoSil ApS © 2026

Timing Checkers

- Signal behavior over time
  - With SystemVerilog then SVAs would be used
  - We do not have SVAs
  - Implement by either
    - Use a Verilog module and behavioral code for the checker
    - Access the signals from PyUVM and impement a uvm_component which uses PyUVM code to check the signal behavior

```python
import pyuvm
from cocotb.triggers import RisingEdge, FallingEdge, Timer, ReadOnly

class SignalChecker(uvm_component):
    def build_phase(self):
        self.clk = self.get_config_object("clk")
        self.sig_a = self.get_config_object("sig_a")
        self.sig_b = self.get_config_object("sig_b")
        self.start_check = False
        self.check_phase = 0
        self.clk_count = 0

    async def run_phase(self):
        while True:
            await RisingEdge(self.clk)
            await ReadOnly()

           if self.sig_a.value.integer == 1 and not self.start_check:
                self.start_check = True
                self.clk_count = 0
            elif self.start_check:
                self.clk_count += 1
                if self.clk_count == 5:
                    if self.sig_b.value.integer == 1:
                        print("Assertion Passed.")
                    else:
                        print("Assertion Failed.")
                    self.start_check = False
```

Timing Checker Example

- Checks that a is asserted 5 clks after a is asserted

SyoSil ApS © 2026

Timing Checkers

- Event Checker
  - Checks that
    - First **start _event** occurs
    - Then **end _event** occurs
  - No double start or end event
  - No end event without start event

```
import pyuvm
from cocotb.triggers import Event

class EventChecker(uvm_component):
    def build_phase(self):
        self.start_event = pyuvm.uvm_event("start_event")
        self.end_event = pyuvm.uvm_event("end_event")
        self.last_event = None  # To keep track of the last event

    async def run_phase(self):
        while True:
            # Wait for either start_event or end_event
            await pyuvm.uvm_any(self.start_event.wait(), self.end_event.wait())
            if self.start_event.triggered:
                if self.last_event == 'start':
                    self.uvm_error("EventChecker", "Two start_events in a row")
                self.last_event = 'start'
                self.start_event.reset()
            elif self.end_event.triggered:
                if self.last_event is None:
                    self.uvm_error("EventChecker",
                                   "end_event occurred without a prior start_event")
                elif self.last_event == 'end':
                    self.uvm_error("EventChecker", "Two end_events in a row")
                self.last_event = 'end'
                self.end_event.reset()
```

Timing Checker Example

- Checks that **start_event** is before **end_event**
- **end_event** cannot occur with a **start_event**

SyoSil ApS © 2026

Where to Apply Checkers?

SyoSil ApS © 2026

Where to Apply checkers?

- **Reference Model**
  - When functionality of the device needs to be modelled and a golden reference is needed
  - Typically, one instance for a module test bench
  - Handling all outputs
- **Score board**
  - When having a stream of transactions to compare
  - Typically, one instance for a module test bench
  - Multiple instances can also occur
- **Transaction Checkers**
  - When a stream of transactions needs to follow a certain pattern
  - Can be either specific for a test bench or implemented within a UVC
- **Timing checkers**
  - When timing of signals or more abstract events etc needs to be checked
  - Can be either specific for a test bench or implemented within a UVC
    - Typically as protocol checks

SyoSil ApS © 2026